High Level Design (HLD)

Satellite Image Threat Detection - System Architecture & Design

Project: Satellite Image Threat Detection (SITP)

GitHub: github.com/harshal3558/Satellite-Image-Threat-Detection

--- High Level Design (HLD)

Satellite Image Threat Detection (SITP)

1. Overview

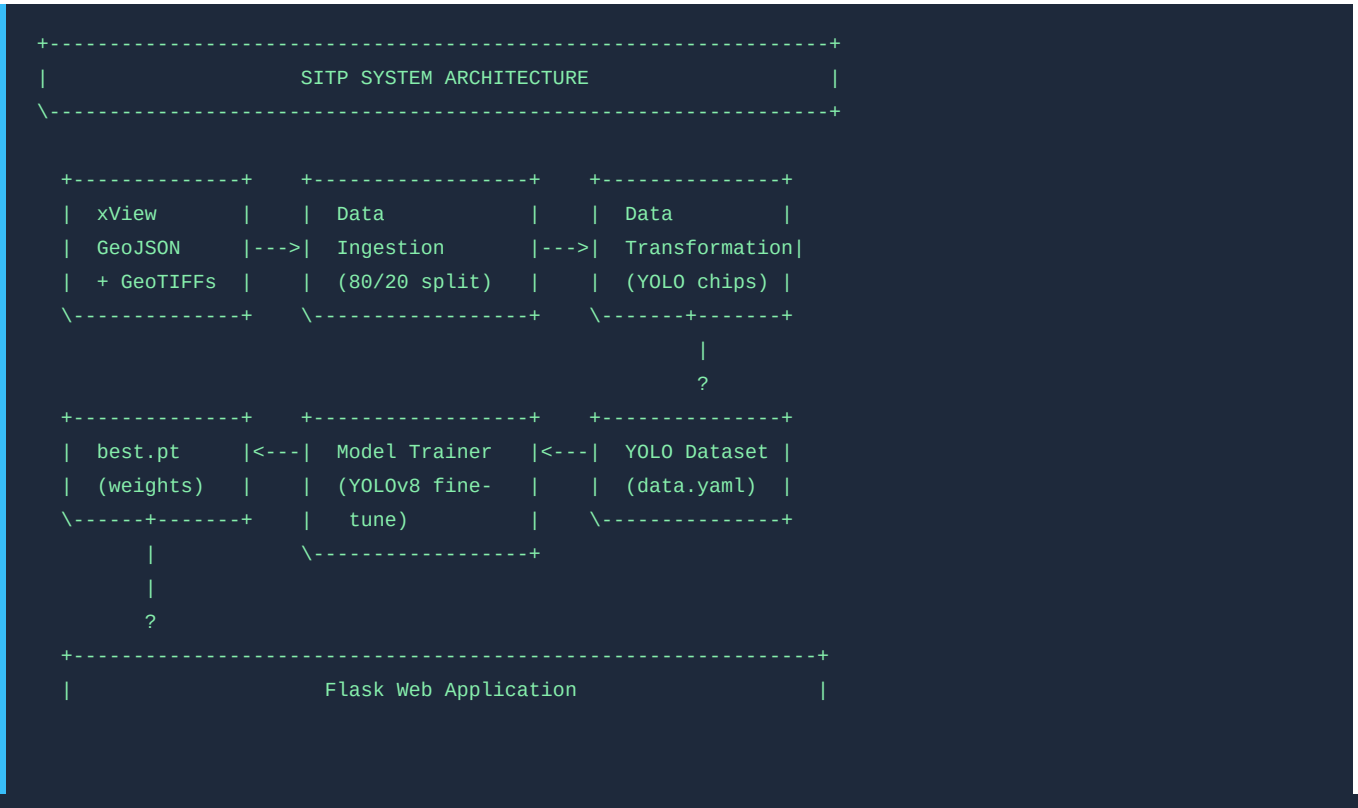
The Satellite Image Threat Detection (SITP) system is an end-to-end Machine Learning pipeline and web application that ingests large-format satellite imagery (GeoTIFF), preprocesses it into training chips, fine-tunes a YOLOv8 object detection model, and performs tiled inference to identify threat-class objects in new satellite images.

Dataset: xView Dataset - multi-class object detection over high-resolution satellite imagery, annotated via GeoJSON.

2. Goals & Objectives

Goal	Description
Automated Threat Detection	Identify and localize threat object
Scalable Preprocessing	Handle large GeoTIFF images via sli
Reproducible Training	Seed-controlled, configurable pipel
Interactive Inference	Web interface for real-time upload
Containerized Deployment	Docker support for portable deploym

3. System Architecture



```
|  User uploads .tif --> Tiled Inference --> NMS --> Results  |
\------------------------------------------------------------+
```

4. Major System Components

4.1 Training Pipeline

```
main.py
  \-- TrainingPipeline.run()
        +-- Stage 1: DataIngestion
        |     +-- Parse xView GeoJSON
        |     +-- Validate images on disk
        |     \-- 80/20 image-level train/val split
        |
        +-- Stage 2: DataTransformation
        |     +-- Sliding-window tiling (512×512, stride=364)
        |     +-- YOLO label format conversion
        |     +-- Albumentations augmentation (train only)
        |     \-- Write data.yaml + class_mapping.json
        |
        \-- Stage 3: ModelTrainer
              +-- Load YOLOv8m pretrained weights
              +-- Fine-tune on YOLO chips
              \-- Save best.pt checkpoint
```

4.2 Inference Pipeline

```
application.py (Flask)
  \-- POST /
        +-- Receive .tif upload
        +-- PredictPipeline.predict()
        |     \-- ModelMonitoring.predict_large_image()
        |           +-- Tile the GeoTIFF (512×512, overlap=100px)
        |           +-- Run YOLOv8 on each tile
        |           +-- Translate tile coords -> global coords
        |           \-- Batched NMS to remove duplicates
        +-- Render annotated image (base64 JPEG)
        \-- Return detections + statistics
```

5. Data Flow Diagram

```
[xView GeoJSON]          [GeoTIFF Images (.tif)]
      |                          |
      ?                          ?
  Parse features             Validate on disk
  (image_id, bbox,           (filter missing)
   type_id)
      |                          |
      \----------+---------------+
```

```
        Image-Level Split
        (80% train / 20% val)
               |
               ?
        Sliding-Window Tiling
        (512×512 chips, stride=364)
               |
        +------+---------------+
        ?                      ?
  Train Chips              Val Chips
  + Augmentation           (no augmentation)
  (CLAHE, flips,
   brightness)
        |                      |
        \----------+-----------+
                   ?
            YOLO-format dataset
            (images/ + labels/ + data.yaml)
                   |
                   ?
            YOLOv8m Fine-tuning
            (50 epochs, batch=8, imgsz=512)
                   |
                   ?
               best.pt
                   |
                   ?
          Tiled Inference on new .tif
                   |
                   ?
            Global NMS -> Detections
                   |
                   ?
           Web App Visualization
```

6. Technology Stack

Layer	Technology	Purpose
Language	Python 3.10+	Core implementation
Object Detection	YOLOv8 (Ultralytics)	Model architecture
Geospatial I/O	Rasterio	GeoTIFF reading
Image Processing	OpenCV, NumPy	Chip manipulation, visualization
Augmentation	Albumentations	Training-time augmentations
Geometry	Shapely	Visibility/intersection calculation
Web Framework	Flask	HTTP server & routing
Data Processing	Pandas	Annotation DataFrame management
Deep Learning	PyTorch (via Ultralytics)	Model training backend
Containerization	Docker	Portable deployment
Logging	Python logging	Pipeline audit trail

7. Environment Detection

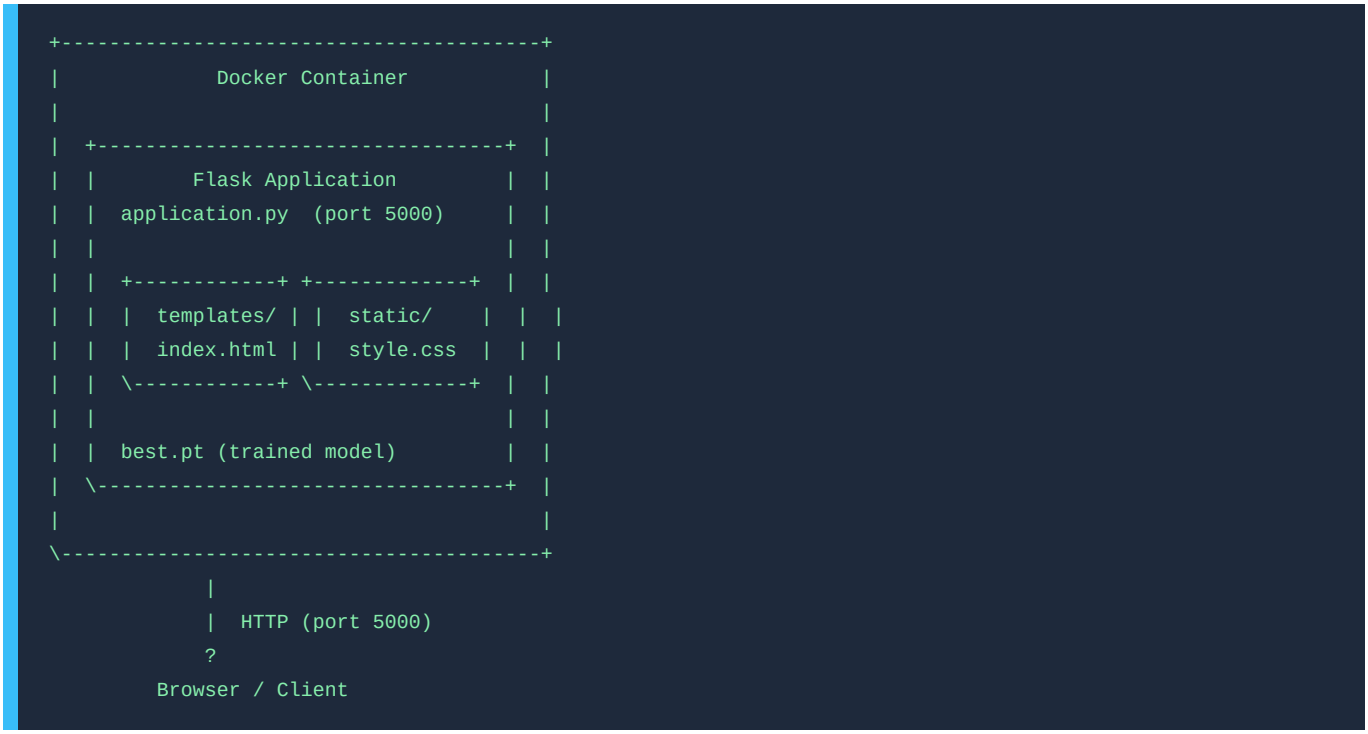
The system auto-detects the execution environment:

Environment	Image Directory	Output Directory
Kaggle	/kaggle/input/.../train_images/	/kaggle/working/xview_yolo/
Local	./data/train_images/train_images/	./xview_yolo/

8. Key Design Decisions

Decision	Rationale
Image-level train/val split	Prevents data leakage from chips of
Sliding-window tiling	Handles large GeoTIFFs that exceed
8% background chip retention	Improves model specificity by expos
0.4 visibility threshold	Ensures only well-visible objects a
NMS on global coordinates	Deduplicates detections that span t
Seed-controlled randomness	Ensures reproducible splits and aug

9. Deployment Architecture



10. Non-Functional Requirements

Requirement	Detail
Max Upload Size	500 MB per request
Supported Formats	.tif, .tiff (GeoTIFF only)
Model Inference	Tiled (512×512 tiles, 100px overlap

Visualization	Downscaled to max 1024px on longest
File Cleanup	Uploaded files are deleted after in
Thread Safety	UUID-prefixed filenames prevent upl